

Construindo uma CPU - MIPS (parte 2)

Pipelining

Duração das instruções

- Qual tipo de instrução nos toma mais tempo?
 - Instruções que lidam com a memória, principalmente loads
 - Passam por cinco unidades funcionais em série
- Qual tipo de instrução é mais rápido?
 - Instruções do tipo jumps
 - Após a memória de instruções, basta fazer o shift left do imediato, e concatenar com o PC
- Temos instruções que em teoria levam tempos diferentes para serem executadas

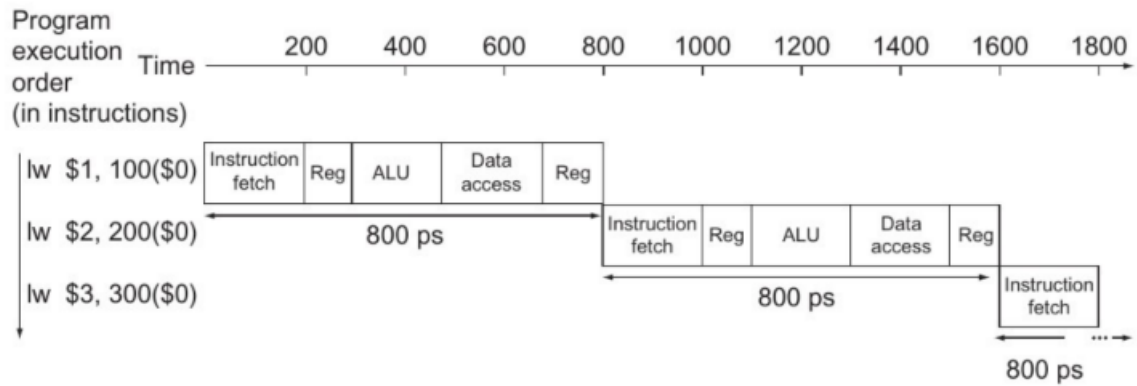
Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Ciclo único

PROBLEMAS DO CICLO ÚNICO

- Componentes de estado são sincronizados por um sinal de clock
- O clock deve ser longo o suficiente para o tempo da instrução ser executada
- Devemos então considerar o pior caso, e ter um período de clock de 800ps
 - Esperar para carregar a próxima instrução
 - Se a instrução “não necessita” de todos os 800ps, jogamos tempo fora

ORDEM DE EXECUÇÃO EM CICLO ÚNICO

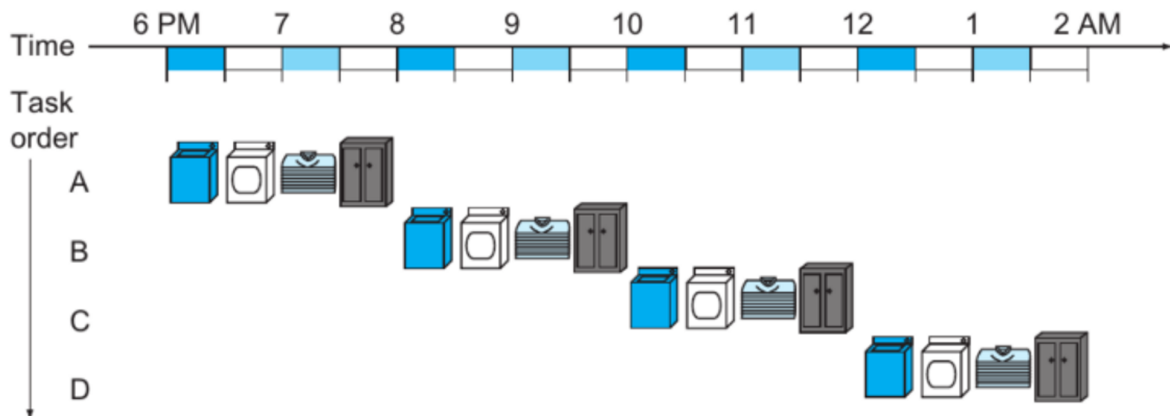


- Processadores extremamente simples podem usar ciclo único
- Mas se adicionarmos mais complexidade ao nosso circuito, um tamanho de ciclo que considera o pior caso se torna impraticável

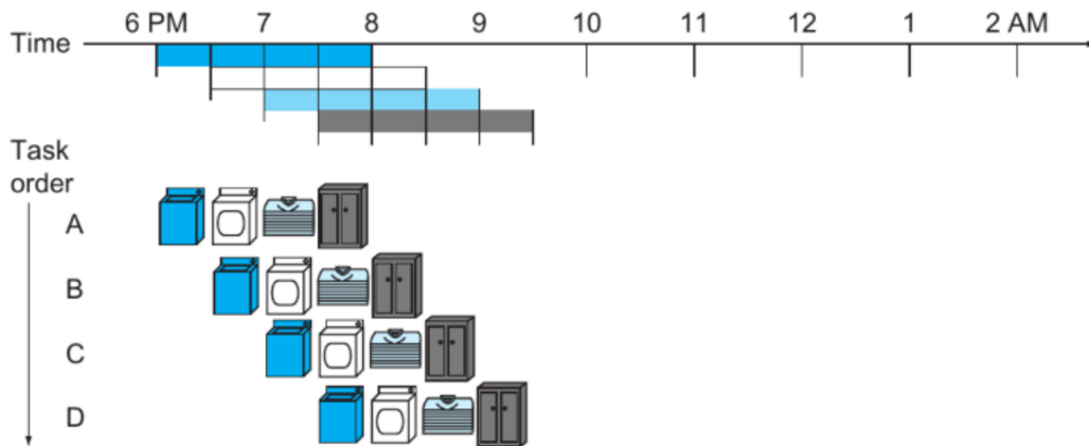
Pipelining

- É utilizada em praticamente todos os processadores atuais
- Transformamos o processador em uma “linha de montagem”

ANALOGIA: LAVANDO ROUPAS SEM PIPELINING



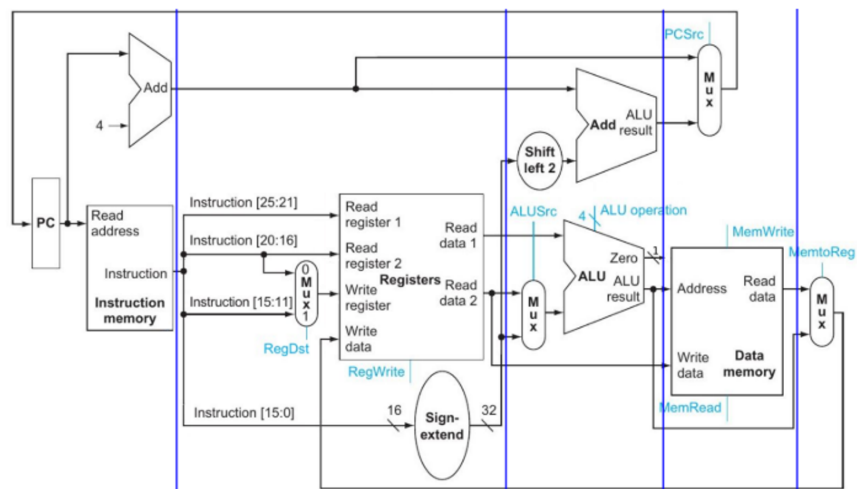
ANALOGIA: LAVANDO ROUPAS COM PIPELINING



- Onde está o ganho de tempo?
 - Temos múltiplas trouxa sendo “processadas” em paralelo, cada uma em um estágio
- Tempo de execução de uma instrução “não muda”
- Tempo gasto para uma instrução individual também pe chamado de **latência**
- Se olharmos um período de tempo suficientemente longo, o número de tarefas completas nesse tempo é muito maior
 - A **vazão (throughput)** pode aumentar até n vezes

CPU MIPS

- Estágios:
 - Instruction fetch (IF): Buscar a instrução na memória
 - Instruction decoding (ID): Ler os registradores enquanto a unidade de controle a decodifica
 - Execution (EX): Executar a operação cálculo do enredoço
 - Memory Access (MEM): Acessar a memória de dados
 - Write Back (WB): Escrever o resultado num registrador



ESTÁGIOS DO PIPELINE

- Nosso processador pode ser dividido em 5 estágios
 - A quantidade de estágios depende diretamente da arquitetura do processador

AJUSTANDO O TEMPO DE CLOCK

- **Sem pipeline**, somamos todos e ajustamos para o pior caso (800ps)
- **Com pipeline**, ajustamos para o estágio de pior caso
 - Os estágios mais demorados precisam de 200ps
 - Logo, o tempo de clock deve ser de no mínimo 200ps

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

- Na prática, nosso ganho vai ser

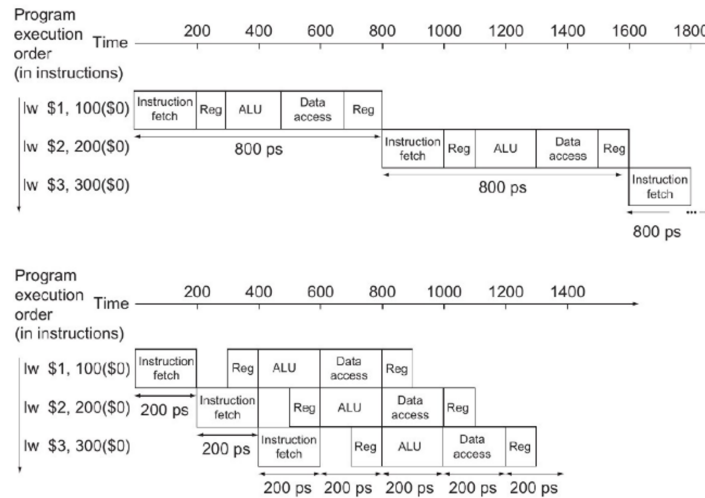
$$\text{proporção do aumento de vazão} = \frac{\text{Tempo do ciclo sem pipeline}}{\text{Tempo do ciclo para a unidade mais lenta}}$$

- Em nosso processador MIPS com os números do exemplo:

$$\text{proporção do aumento de vazão} = \frac{800}{200} = 4$$

- 5 estágios, potencial para até 5 vezes mais rápido
- Porém, neste caso é 4 vezes mais rápido

Ciclo único vs Pipelining



Hazards

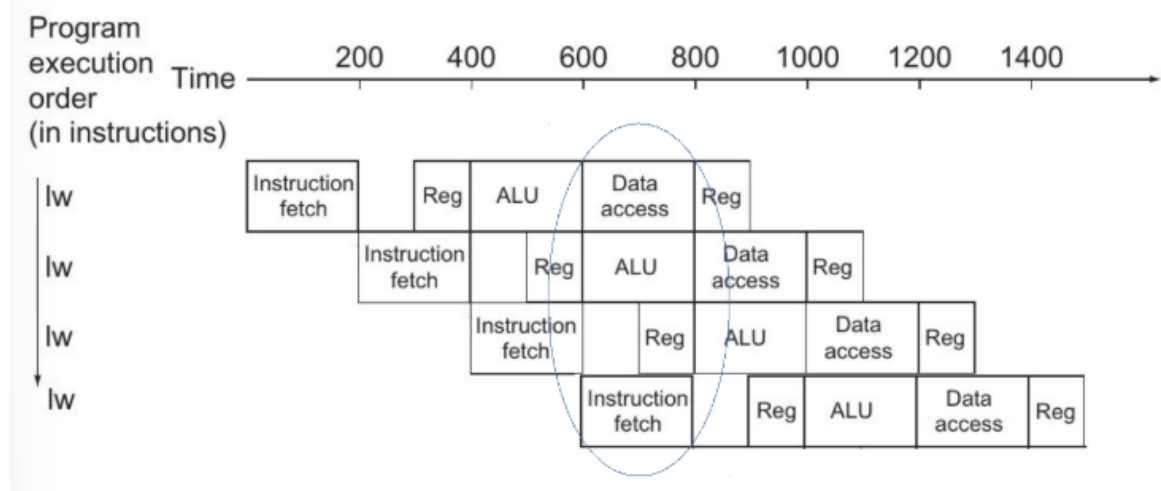
- Ao implementarmos o conceito de pipeline, criamos diversos problemas e complexidades

Hazard: risco ou perigo

- Impedem que a próxima instrução continue sua execução no ciclo do clock seguinte

Hazards Estruturais

- Em um hazard o hardware não pode manter duas instruções no pipeline, pois elas estão competindo por algum componente
- Se tivéssemos uma única memória, para dados e instruções
 - Enquanto uma instrução está sendo carregada, outra está tentando escrever na memória ao mesmo tempo
- Se tivéssemos uma única memória (para instruções e dados), em algum momento no pipeline duas instruções poderiam competir pelo acesso a essa memória
- Se nosso banco de registradores não permitisse ler e escrever ao mesmo tempo
 - Não podemos ter ID e WB ao mesmo tempo

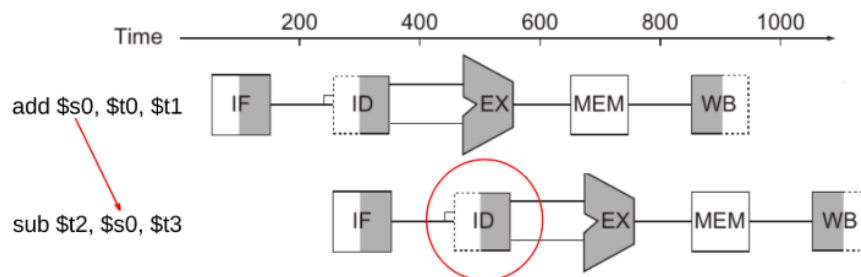


- Nossa implementação do MIPS não sofre com hazards estruturais
 - Os componentes que poderiam conflitar em determinado instante já estão duplicados
 - Um hazard estrutural muitas vezes é corrigido criando-se cópias das unidades funcionais, para instruções em diferentes estágios do pipeline utilizem diferentes cópias da unidade funcional

Hazards de Dados

Considerando as instruções abaixo:

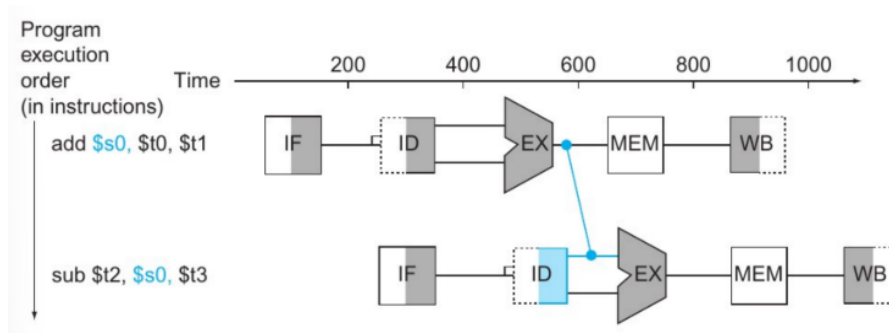
```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```



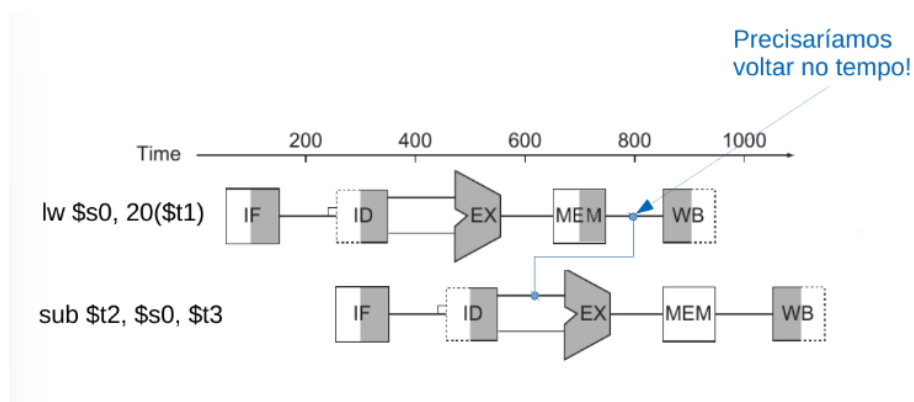
- O resultado da operação é escrito em \$s0 somente no estágio WB, mas os valores dos registradores estão sendo buscados em ID, e \$s0 ainda não está “pronto”
- Uma instrução depende da outra

FORWARDING

- Também conhecido como bypassings
- “Emprestamos” o resultado de uma unidade antes da operação ter sido completada (escrita na memória ou banco de registradores)
- Observe no exemplo anterior que o resultado já está pronto após a unidade EX, ele só não foi escrito no registrador ainda
 - Sendo assim, podemos tomar esse resultado e utilizar como entrada para o EX da próxima instrução

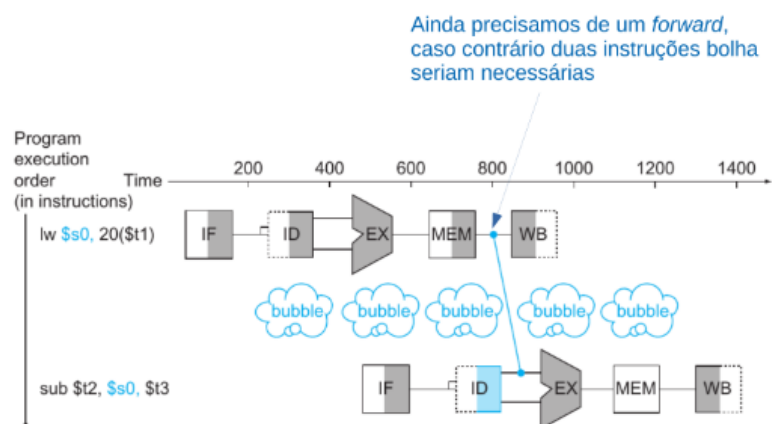


Exemplo: Uma sequência de instruções onde o forwarding não vai resolver o problema



PIPELINE STALL (BOLHAS)

- Um hazard de dados pode exigir o atraso da execução da próxima instrução
 - Ao detectar isso, o processador deve injetar uma instrução bolha entre as instruções que estão com o hazard
 - Somente gasta tempo, para que o resultado necessário possa ser computado
- Geramos um pipeline stall
 - Tivemos que parar nossa “linha de montagem” por um momento para então continuar



- Usamos um stall e foward
 - Sem foward, seria necessário pelo menos dois stalls
 - Considerando que com WB e ID no mesmo clock, Wb termina antes de ID “pegar” os valores no registrador

```
addi $t5, $t5, 10
lw $s0, 20($t1)
sub $t2, $s0, $t3
```

- para evitar evitar o stall o compilador pode reordenar as instruções ao gerar o código de máquina, e nesse caso isso não afetaria o resultado do programa

```
lw $s0, 20($t1)
addi $t5, $t5, 10
sub $t2, $s0, $t3
```

O papel do compilador

- Compiladores sofisticados, como o GCC, tentam de todo modo criar uma ordem nas instruções de modo a evitar stalls no pipeline
 - A CPU pode por conta própria executar as instruções em uma ordem diferente para evitar stalls
 - utiliza técnicas como buffers de reordenação e renomeação de registradores

Hazards de Controle

- O pipeline funciona encadeando as instruções, uma após a outra, em nossa “linha de montagem”
 - Mas e se não sabemos qual a próxima instrução

```
add $4, $5, $6
beq $1, $2, 1
lw 3, 300($0)
or $7, $8, $9
```

- Até o beq terminar o estágio de EX não sabemos se executamos o lw

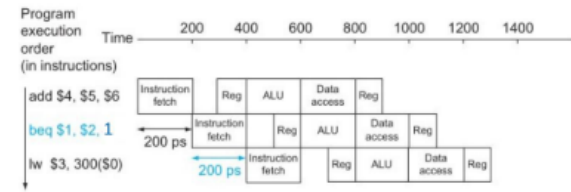
SOLUÇÃO

- Sempre considerar que o desvio não foi tomado
- Quando a instrução de desvio terminar de executar
 - Se o desvio realmente não devia ser tomado, tudo continua normalmente
 - Se descobrirmos que o desvio deveria ser tomado, descartamos as instruções que estão no pipeline e carregamos as instruções a partir do endereço correto.
 - Injetar bolhar no meio do caminho

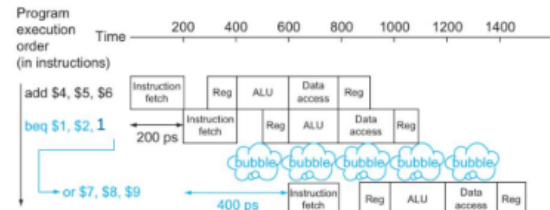

```

1 add $4, $5, $6
2 beq $1, $2, 1
3 lw 3, 300($0)
4 or $7, $8, $9

```



Se a CPU concluir que o desvio deveria ser tomado, precisamos descartar as etapas do lw processadas, trocar por bolhas, e colocar a instrução correta na fila do pipeline.



- Podemos ainda adicionar unidade de predição dinâmica de desvios
 - A CPU tenta “adivinhar” se o desvio vai ser tomado ou não
 - Mantemos uma tabela com o endereço da instrução, e se o desvio foi tomado na última vez que executamos essa instrução ou não
 - Especialmente útil para desvios em loops

Caminho de Dados com Pipeline

Problemas nas fronteiras

Considere o seguinte programa:

```

lw $2, 100($0) # No estágio ID
lw $3, 200($0) # No estágio IF
lw $4, 300($0)

```

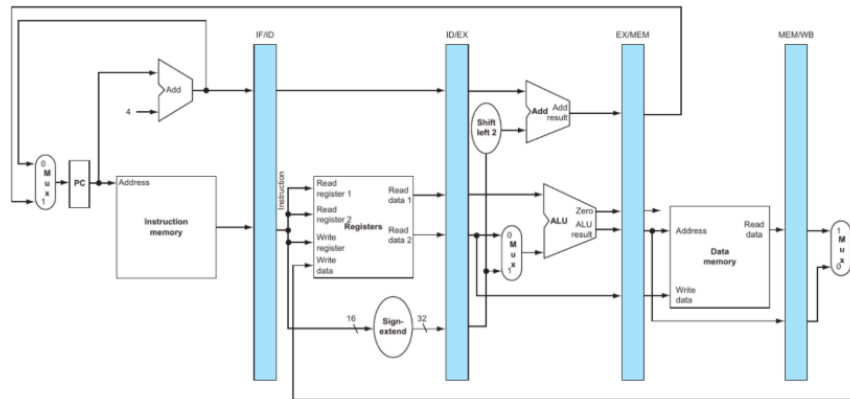
- PC é enviado no primeiro estágio, de onde vai sair a primeira instrução
- PC (atualizado) é enviado no primeiro estágio, de onde vai sair a segunda instrução
- Primeira instrução é enviado para o estágio ID, para ler os registradores

Problema: no estágio ID, desejamos a instrução que havia sido carregada no estágio IF no ciclo de clock anterior (lw \$2, 100(\$8)), mas agora só temos o sinal da instrução atual em IF

- O problema se repete nos demais estágios, conforme as instruções “caminham” em nosso fluxo

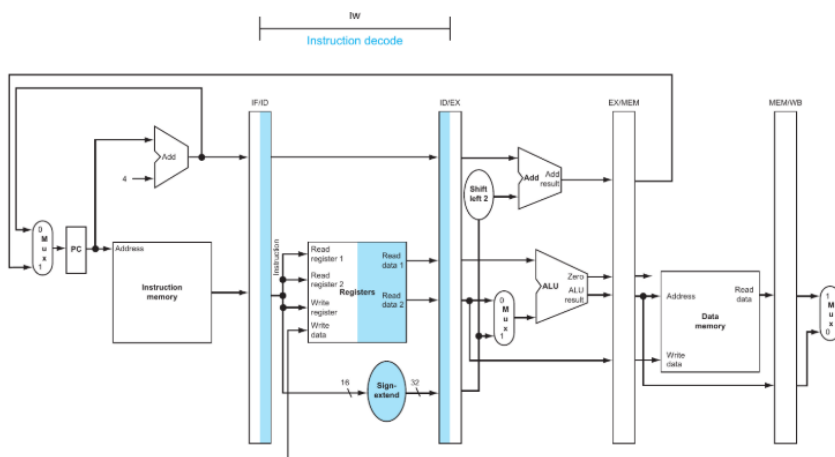
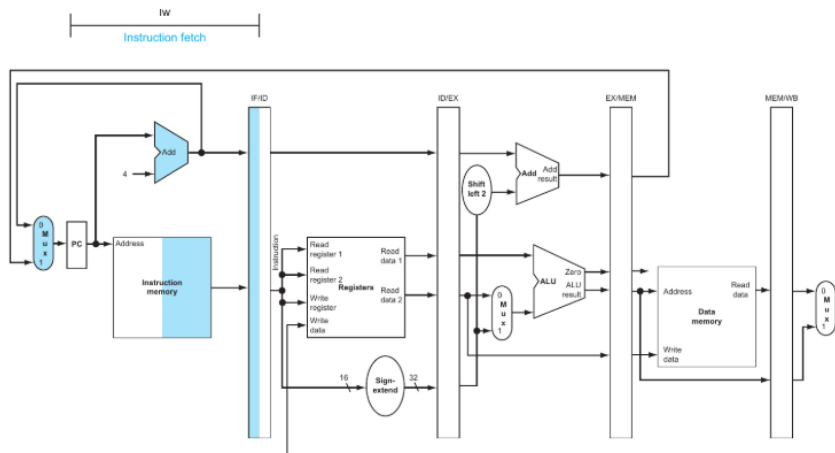
Resolvendo:

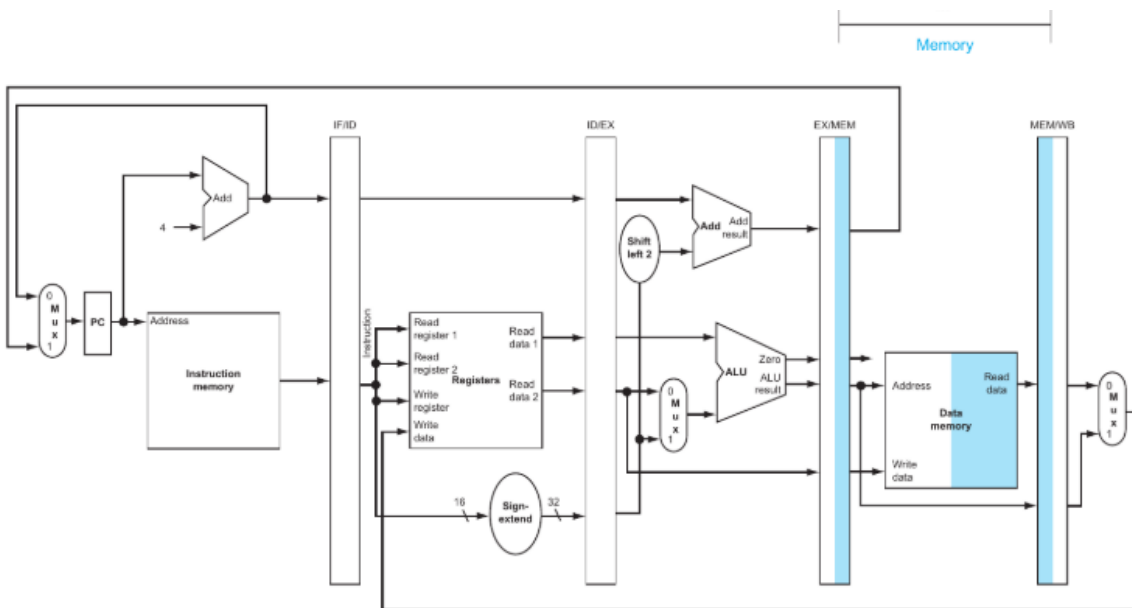
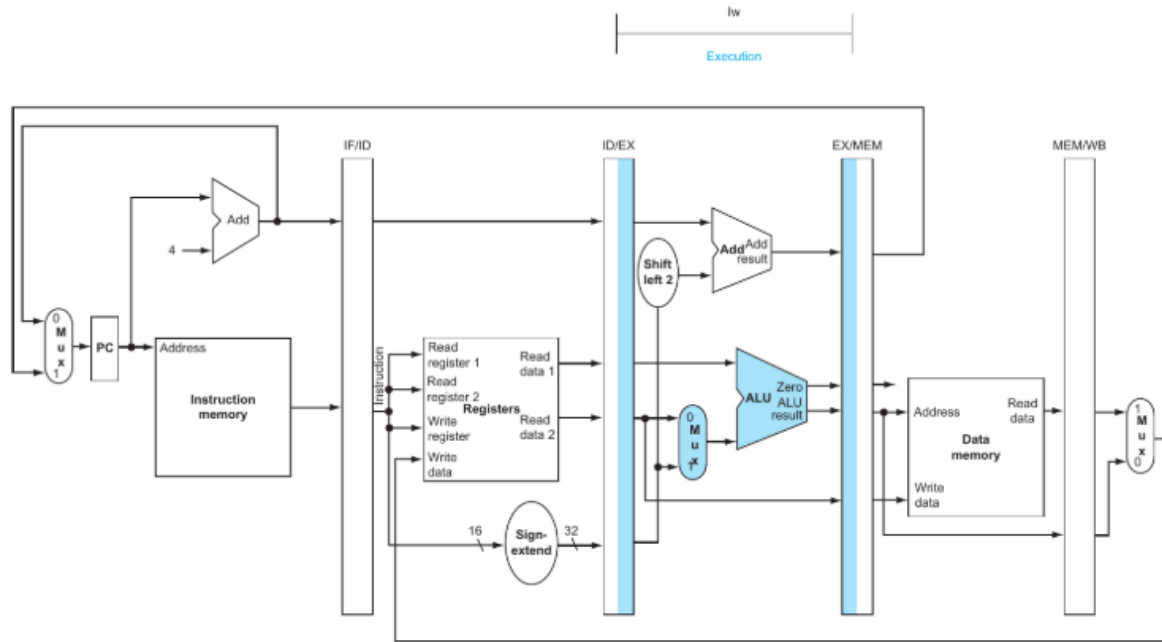
- Utilizamos registradores de pipeline
 - Salvamos toda a informação que é pertinente para o próximo estágio do pipeline no próximo ciclo de clock
 - Continuando com a analogia da lavanderia, teríamos cestos de roupas para armazenar a roupa antes de passar para o próximo estágio

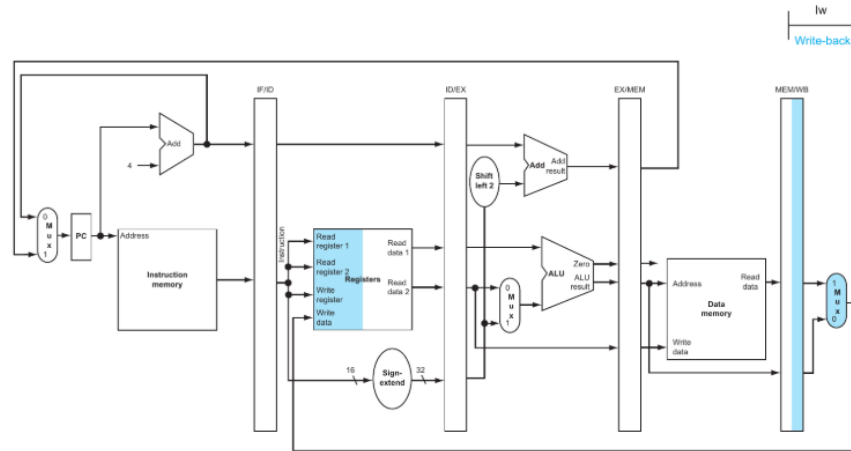


Exemplo do fluxo do lw no pipeline

- Quando a área sombreada dos registradores é a esquerda, os registradores estão sendo escritos
- Quando a área sombreada dos registradores é a direita, os registradores estão sendo lido

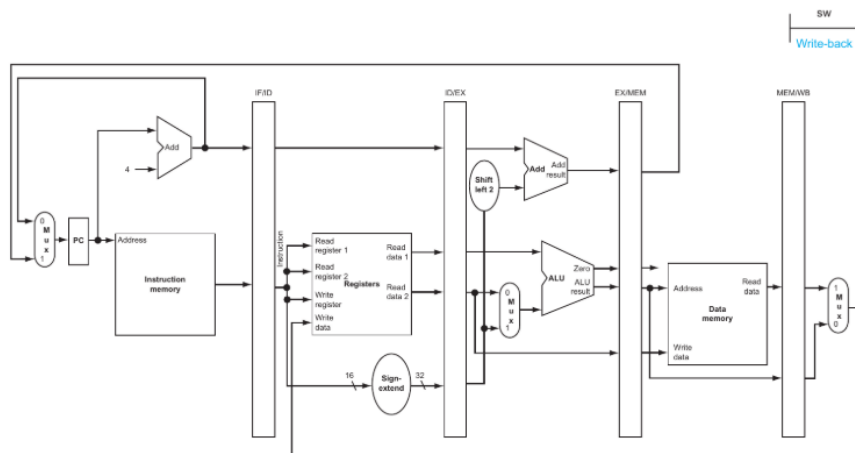
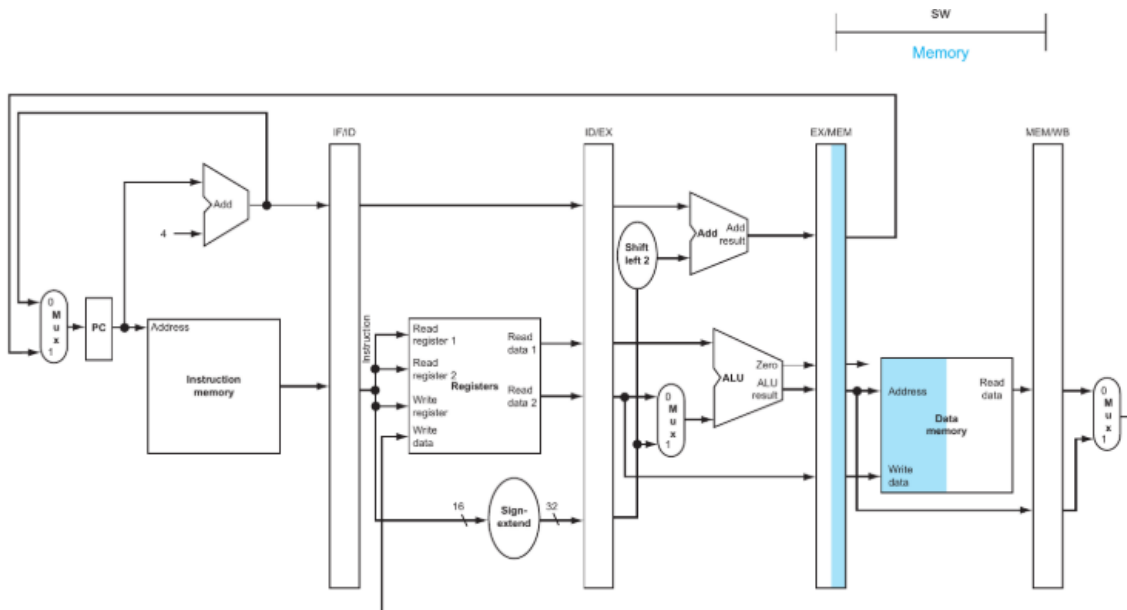






Exemplo do fluxo do sw no pipeline

- Os primeiros estgios do sw so os mesmos do lw

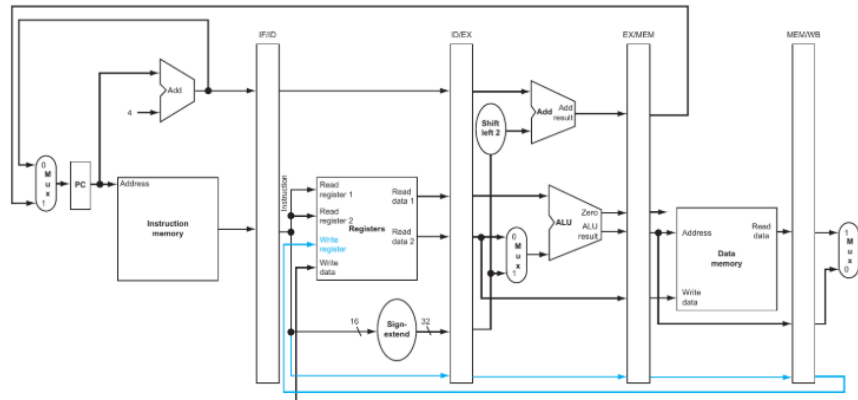


BUG

- Qual o problema com o registrador a ser escrito? q
- Dados de um estágio (uma instrução anterior), endereço de outro estágio (instrução mais “á frente”).

Resolvendo BUG

- Precisamos salvar pelo menos o endereço do registrador de escrita até o estágio WB. Por enquanto esse valor está se perdendo em nosso pipeline, e estamos escrevendo no registrador endereçado pela instrução que se encontra no estágio ID, e não pela instrução do estágio WB.



Exercício

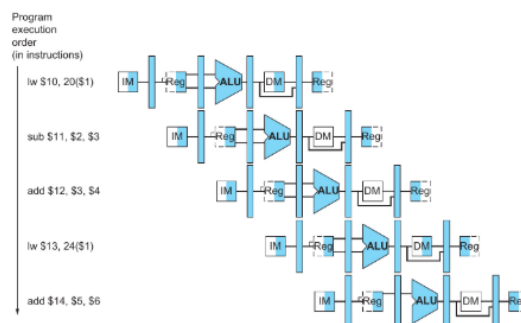
- Considere as instruções

```

1 lw $t0, 20($t1)
2 sub $t1, $t2, $t3
3 add $t2, $t3, $t4
4 lw $t3, 24($t1)
5 add $t4, $t5, $t6

```

Faça um diagrama de múltiplos ciclos de clock para essas instruções (veja o exemplo para o lw). Em cada componente do diagrama, pinte-o de acordo com o exemplo para indicar que a unidade está sendo utilizada naquele estágio.



Adicionando sinais de controle

- Os sinais de controle são (por enquanto) os mesmos que na máquina de ciclo único q
- O sinal pode ser definido já no estágio ID q
- Podemos simplesmente ligá-los em nossa CPU? Problemas? $\dot{a}$
- Diferentes sinais são utilizados em diferentes estágios do nosso pipeline

- Assim como os dados, devemos salvar os sinais de controle à ... também nos registradores de Pipeline

Exceções, Interrupções e I/O

- Podem mudar o fluxo normal do programa

Exceção: Algum evento interno inesperado que ocorreu dentro do processador

- Ex: Overflow, instrução inválida, programa chama o S.O. (syscall)

Interrupção: Algum evento inesperado gerado externamente

- Ex: Usuário digitou algo no teclado, Solicitação de E/S

No Mips tudo é exceção, a interrupção é um tipo especial de exceção

Tratando uma Exceção (Sistema operacional)

- Considerando o caso do overflow aritmético

Passos básicos

1. Salvar o endereço da instrução que causou a exceção
 - a. Salva o endereço do PC no (EPC: Exception PC)
2. Salvar algum código informando o que causou a exceção
 - a. Registra cause no MIPS (indica a causa da exceção mais recente)
 - i. Código 1 indica overflow já o 2 significa instrução inválida
3. Transferir a exceção para o sistema operacional
4. O S.O verifica o registrador cause para definir o que houve
 - a. O S.O tem a opção de tratar a exceção do programa a partir do ponto salvo em EPC, ou terminar o programa

Tratando uma Exceção (Interrupções Vetorizadas)

- Outra forma de tratar exceções é eliminar o registrador cause e transferir o controle para um endereço específico, dependendo da fonte da exceção

Tipo da exceção	Endereço de desvio
Instrução indefinida	0x80000000
Overflow aritmético	0x80000180
Requisição de E/S do usuário	0x80000360
...	...

Vantagens e desvantagens

- Precisa de mais hardware
- Menor flexibilidade
- Tratamento mais rápido

x86 utiliza interrupções vetorizadas enquanto o MIPS usa o registrador case, mas apenas para (falta de páginas) utiliza interrupção vazia

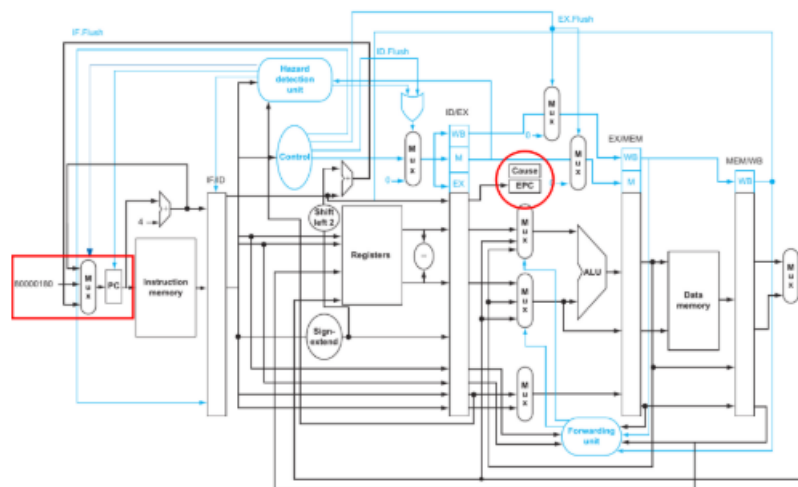
Tratando uma Exceção no pipeline

- Uma exceção é tratada como hazard de controle (o hardware não pode admitir a combinação de instruções que queremos executar no mesmo clock)

Passos básicos (se a exceção foi causada internamente)

1. Salvamos as informações
2. Descartamos as instruções anteriores que já estão no pipeline
3. Redirecionamos para o S.O (ou uma rotina de tratamenti qualquer)

obs: Mt parecido com o que ocorre no desvio condicional, a diferença é que é necessário salvar algumas informações, e o endereço de desvio é fixo (0x80000180)



Obs: Note que estamos salvando PC+4 em EPC. O S.O deve levar isso em cosideração ao tratar umaexceção nesse processador MIPS

Comunicando-se com dispositivos

Todo dispositivo de E/S é composto por pelo menos dois componentes:

- **Parte mecânica:** HD, teclado
- **Parte eletrônica:** Possui pelo menos alguns registradores para indicar o que o componente está fazendo, ou precisa fazer

Espaço de E/S

- Controladores especializados podem dar um número único para cada registrador de cada dispositivo de E/S conectado
 - Número de portas de E
- Podemos desenvolver instruções para nossa CPU, que recebe um número de porta e o valor escrito nela
 - Exemplos (instruções estilo x86)

- in REG, PORTA: lê o conteúdo da porta especificada e armazena em REG
- out PORTA, REG: escreve o conteúdo de REG para a porta especificada

E/S Mapeada em Memória

- Controlador dá um endereço de memória para cada registrador de E/S
 - Ao ler e escrever nesse endereço de memória, o controlador detecta, e redireciona os dados para o registrador do dispositivo
 - Nada é realmente li/escrito na memória principal da máquina
- Chamamos de E/S mapeada em memória q
 - Agora instruções lw e sw podem ser usadas para escrever na memória, e também para nos comunicar com os demais dispositivos de E/S
 - Precisamos “apenas” de um controlador que intercepta o sinal da CPU, e redireciona para o local correto

| O MIPS utiliza o E/S mapeada em memória, enquanto x86 utiliza ambas as técnicas

Enviando sinais ao processador

- Um dispositivo E/S pode enviar um sinal ao processador
- Quando o processador recebe um sinal externo, ele gera um interrupção (exceção gerada por um dispositivo externo)
- O processador pode também ler o barramento externo para identificar o dispositivo que gerou a interrupção
- Quando uma interrupção é gerada, o processador pode terminar de executar as instruções que já estão no pipeline, antes de redirecionar o S.O
 - A exceção não é interna e não tem nada que impeça as instruções que estão na metade do caminho de terminarem
- O processador armazena a interrupção e redireciona para o S.O., que então lida com a interrupção da mesma forma que uma exceção qualquer

Arquiteturas e Abstrações

Arquitetura de Von Neumann

Programa armazenado: O programa é armazenado na memória do computador juntamente com os dados

Ciclo de busca-decodificação-execução:

1. A CPU busca a próxima instrução da memória utilizando um contador de programa
2. A instrução é decodificada para algo que uma ALU possa entender
3. Os operandos necessários são carregados
4. A ALU executa a operação e o resultado é armazenado na memória

Arquitetura composta por:

- CPU com unidade de controle, ALU, registradores e contador de Programa
- Uma memória principal que armazena o programa e seus dados e sistema de E/S

- O fato da memória armazenar tanto o programa e seus dados pode gerar um problema
 - Gera um gargalo, conhecido como Von Neumann bottleneck (Gargalo de Von Neumann)
- Capaz de executar instruções sequencialmente
- Um único caminho (lógico ou físico) até a memória principal

Nossa CPU MIPS segue o ciclo de Von Neumann?

- Não exatamente
 - Temos duas memórias separadas (memória de instruções e memória para dados)
- Maioria das CPUs modernas não seguem a arquitetura de Von Neumann de maneira estrita

Arquitetura de Harvard

- Temos memórias para dados e instruções separadas (igual ao MIPS)

Máquinas com Arquitetura Harvard Pura

- Seguem estritamente conceito
- São comuns em microccontroladores (é um “computador” completo em um chip)

Máquinas com Arquitetura Harvard Modificada

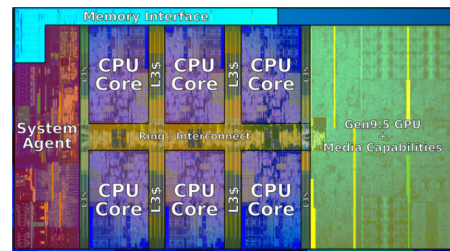
- Relaxa a separação física entre memória de dados e instruções
- Nos níveis mais próximos ao processador, temos arquitetura de Harvard
 - Temos memórias cache separadas para dados e instruções
- Nos níveis de memória mais distantes do processador, temos uma máquina de Von Neumann
 - Os dados são acessados por um único barramento até a memória

Arquiteturas Paralelas

- Computadores Paralelos ¶
 - Existem diversos níveis de paralelismo:
 - Pipelining
 - Processadores superescalares
 - Processadores multicore
 - Multiprocessadores
 - Grids (heterogêneo) ¶
 - Clusters (homogêneo)

Processadores Multicore

- Um processador multicore (com múltiplos núcleos) possui vários núcleos de processamento (processadores) em um único chip
 - Todos os processadores acessam a mesma memória principal por um mesmo barramento



- Por serem paralelos, temos efetivamente múltiplas instruções sendo executadas por diferentes CPUs, ao mesmo tempo:
 - Sendo assim, muitos os classificam como não sendo máquinas de Von Neumann
 - Não processam as instruções de forma sequencial
 - Outros veem essas CPU's como pertencendo a arquitetura de Von Neumann
 - Cada "core" é uma máquina de Von Neumann, cooperando para executar as tarefas

Mix and Match (misturado): Podemos ver uma CPU x86-64 atual então como sendo dependendo do nível de memória uma Arquitetura Harvard e Arquitetura de Von Neumann

CISC

- Complex Instruction Set Computer
- Contém um número (muito) grande de instruções
- Instruções de tamanhos variados
- Instruções complexas, que podem executar "múltiplas coisas" em uma única operação:
 - uma mesma operação busca o dado da memória, realiza uma operação com esse dado, e armazena o resultado na memória

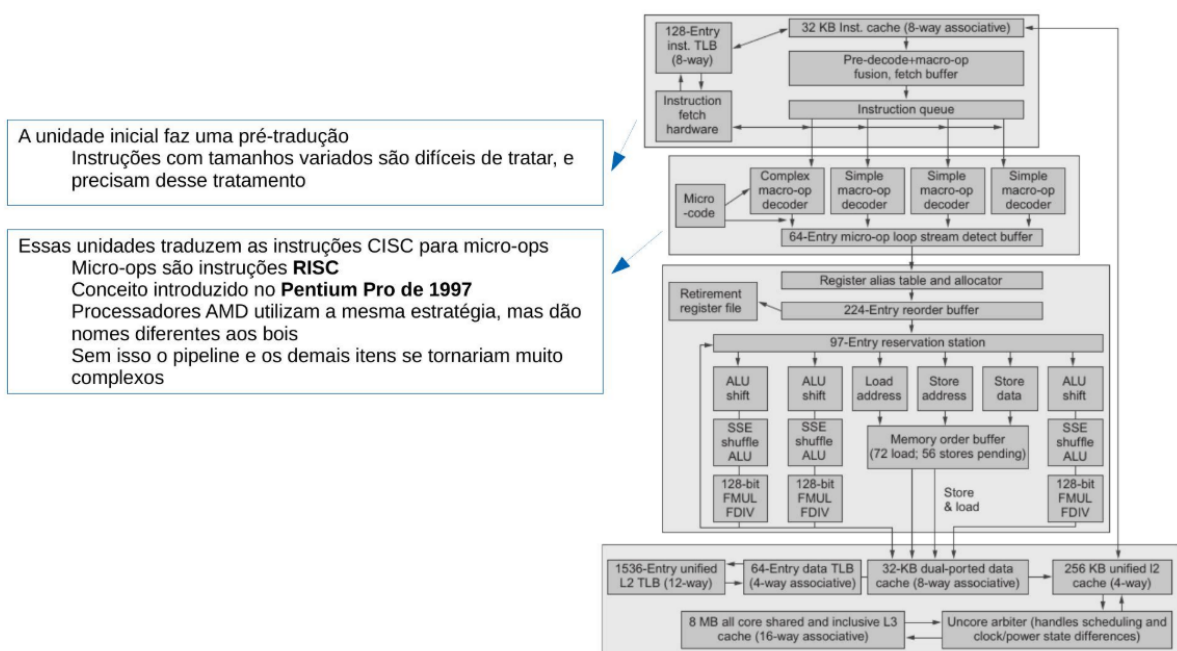
RISC

- Reduced Instruction Set Computer Geralmente:
- Possui um conjunto de instruções reduzido e simplificado
 - Instruções são menos "poderosas"
 - Objetivo é simplificar o projeto e criar instruções que executam mais rapidamente
- Poucos formatos de instruções
- Instruções de tamanho fixo

MIPS é RISC

RISC	CISC
Instruções de tamanho fixo	Instruções de tamanho variado
Muitos Registradores	Poucos Registradores
Instruções de 3 operandos	Instruções com 1 ou 2 operandos
Parâmetros passados via registrador	Parâmetros passados via pilha
Controle hardwired	Controle microprogramado
Pipeline Profundo e simplificado	Pipeline raso e mais complexo
Poucas instruções simples	Muitas instruções sofisticadas e complexas
Somente loads e stores acessam a memória	Muitas instruções podem acessar a memória

Um Intel core i7



Resumindo

- Seu processador é uma casca CISC envolvendo um processador RISC
- Pense em quanto hardware é “jogado fora” só para criar essa casca que traduz de CISC para RISC!

Paralelismo: Conceitos básicos

Mips Estudado

- Temos um paralelismo a nível de instrução
 - Através do pipeline, múltiplas instruções são executadas “ao mesmo tempo”
 - Em estágios diferentes
- Porém
 - Somente uma instrução é enviada a cada ciclo de clock
 - Somente uma instrução (pode ser) completada a cada ciclo

- Ganho está na redução do tempo de cada ciclo de clock
- A instrução opera em apenas um dado
 - Fazem a operação e armazenam o resultado em um registrador

SISD

| A CPU MIPS estudada é um exemplo de uma CPU SISD

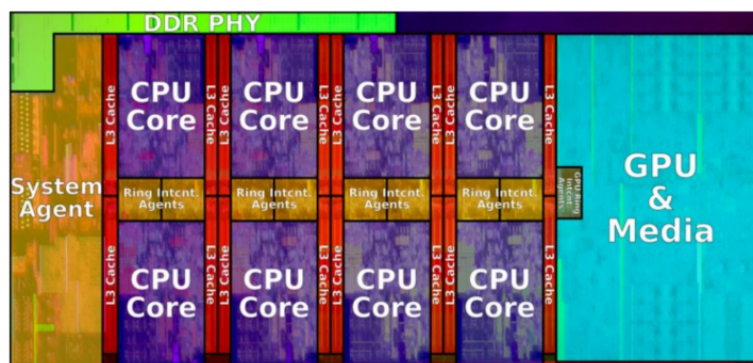
SISD: Single Instruction Single Data

- Uma instrução para um dado
- Processadores que executam uma instrução por vez, e cada instrução é capaz de operar em apenas um dado

Multiprocessadores

- É um processador composto de múltiplos processadores
 - Nossas máquinas x86-64 atuais são exemplos de multiprocessadores

i9-9900k



- Em um sistema multiprocessado, cada processador pode executar uma tarefa independente
 - Paralelismo a nível de tarefa/processo
- Se você escrever um programa sequencial, ele vai usar somente um dos n processadores disponíveis

MIMD

MIMD: Multiple Instruction Multiple Data

- Múltiplas instruções e múltiplos dados
- Multiprocessadores podem ser considerados MIMD
 - Possuem n processadores, cada um operando uma instrução diferente
 - Paralelismo a nível de instrução
 - Cada instrução opera em um dado diferente
 - Paralelismo a nível de dados

MISD

MISD: Multiple instruction single data

- Múltiplas instruções operando em um dado único
- Não existem computadores puramente MISD atualmente

SIMD

SIMD: Single instruction Multiple Data

- Uma instrução operando em múltiplos dados
- Exemplo: uma instrução que soma um valor imediato em múltiplos registradores ao mesmo tempo

Resumo

		Dado	
		Único	Múltiplos
Instrução	Única	SISD: MIPS estudado	SIMD: um único "core" de uma CPU i7 atual
	Múltiplas	MISD: ????	MIMD: Intel i7

SIMD no processador MIPS

- É comum termos de carregar múltiplos endereços de memória para os registradores, somar um imediato, e depois armazenar os resultados
 - Por exemplo:

```
1 lw $s0, 0($t0)
2 lw $s1, 4($t0)
3 lw $s2, 8($t0)
4 lw $s3, 12($t0)
5 addi $s0, $s0, 1
6 addi $s1, $s1, 1
7 addi $s2, $s2, 1
8 addi $s3, $s3, 1
9 sw $s0, 0($t0)
10 sw $s1, 4($t0)
11 sw $s2, 8($t0)
12 sw $s3, 12($t0)
```

Primeira ideia

- Adicionar um `s` no final dos de nossas instruções para indicar que são SIMD

```
1 lws $s0, $s1, $s2, $s3, 0($t0)
2 addis $s0, $s1, $s2, $s3, $s0, $s1, $s2, $s3, 1
3 sws $s0, $s1, $s2, $s3, 0($t0)
```

Problema

- A instrução tem 47 bits e o mips opera com instruções de 32 bits

Criar registradores "grandes"

- ```
lws xmm0, 0($t0)
```

- ```
lws xmm0, 0($t0)
```

```
lws xmm0, 0($t0)
```

- ```
lws xmm0, 0($t0)
```

```
lws xmm0, 0($t0)
```

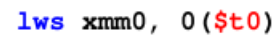
- ```
lws xmm0, 0($t0)
```

```
lws xmm0, 0($t0)
```

- ```
lws xmm0, 0($t0)
```

```
lws xmm0, 0($t0)
```

```
lws xmm0, 0($t0)
```



- ```
lws xmm0, 0($t0)
```

- Adicionar os registradores SIMD no banco de registradores $\mathcal{d}$
 - Novo banco de registradores $\mathcal{q}$
 - Precisamos de 4 ALUS, para que cada ALU faça o cálculo de um dos inteiros $\mathcal{d}$
 - Caso contrário criaríamos uma fila na ALU, executando o cálculo de um inteiro por vez
 - É exatamente essa a estratégia utilizada nas CPUs x86 atuais $\mathcal{q}$
- Anotações
- Um dos conjuntos de instruções especializados do x86-64 para realizar esse tipo de operação é o SSE (Streaming SIMD Extensions)

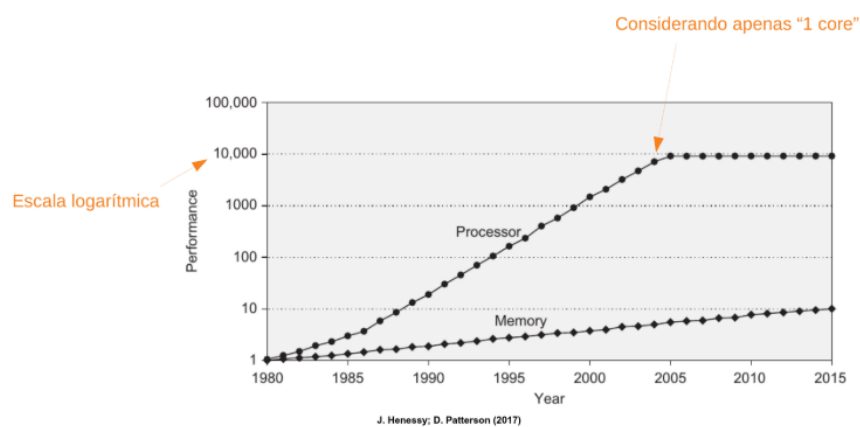
SEE

- SSE do x86-64 funciona utilizando 8 registradores de 128 bits $\mathcal{q}$
 - `xmm0, xmm1, ..., xmm7`
- Dependendo da operação que executamos nesses registradores, podemos operar em paralelo
- As operações SSE exigem que o vetor no qual estamos operando comece em um endereço de memória múltiplo de 16 $\mathcal{q}$
 - Restrição de alinhamento de memória

Hierarquia de Memórias e Cache

- Precisamos de memória para armazenar as instruções dos nossos programas e os dados
 - Além de grande, desejamos uma memória rápida

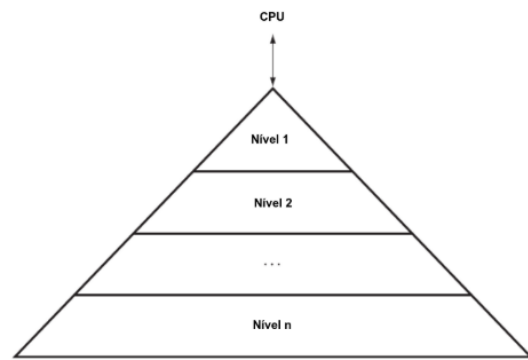
Diferença de desempenho



- Diferença entre a velocidade em que o processador consegue processar instruções versus a velocidade em que a memória consegue suprir essas instruções

Hierarquia de Memórias

- Para amenizar esses problemas, criamos uma hierarquia de memórias, com n níveis
 - A quantidade de níveis vai depender da arquitetura da máquina



Níveis mais próximo da CPU → Velocidade e Custo

Níveis mais distantes da CPU → Capacidade de armazenamento

Computador x86-64 Caches → Memória principal (RAM) → Memória Secundária (persistente): HD ou SSD (ou ambos)

Acesso aos níveis

- Na maioria das implementações, temos uma hierarquia real
 - A CPU acessa os dados apenas no nível 1
 - O nível 1 serve dados para a CPU, e requisita dados do nível 2
 - O nível 2 serve dados para o nível 1, e requisita do nível 3

Memória principal

- A memória principal fica entre os níveis da cache e armazenamento permanente
 - Cache: desejamos velocidade
 - Armazenamento permanente: desejamos mais espaço

Memória cache

- Enquanto nossos computadores hoje têm comumente 16GB de memória principal (DRAM), eles possuem cerca de 12MB de cache (SRAM)
- É comum encontrarmos processadores para servidores com 64MB de cache, mas dificilmente passamos disso

O programador sempre vai assumir que o programa está na memória principal, e não vai programar explicitamente para os demais níveis

Mapeamento

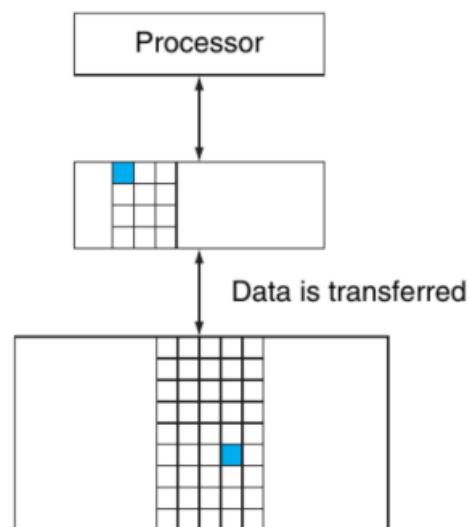
- Como mapear a memória principal para dentro de nossa cache de maneira automática (analisando qual dado é importante)?
 - **Localidade temporal:**

- Se um item da memória é acessado, é muito provável que ele seja acessado novamente num futuro próximo
- Exemplo: quando armazenamos um item na pilha para fazer uma chamada de função, vamos ler ele novamente quando a função terminar
- **Localidade espacial:**
 - Se acessamos um item da memória, é muito provável que seus vizinhos também sejam acessados.
 - Exemplo: Nossos programas são (quase) sequencias, então quando a instrução i é carregada, é provável que $i+1$, $i+2$, ... também sejam úteis

A memória cache é muito menor que a memória principal, por causa disso usamos os dois tipos de localidade, isso com o objetivo de diminuir acesso à memória principal (mais lenta que a cache)

Conceitos de Mapeamento

- Vamo dividir nossas memórias (principal e cache) em blocos
 - Memória chache suporta menor blocos que a principal



Requisitando dados

- Hit (acerto): O dado está presente em um bloco no nível de memória mais alto (cache)
- Miss (erro): Se o dado não está presente, com isso vamos gastar tempo para carregar o bloco

Medidas de performance

- hit rate
 - Fração dos acessos a memória nos quais tivemos um hit q
 - $hitRate = hits / totalAcessos$
- miss rate
 - Fração dos acessos a memória nos quais tivemos um miss q
 - $MissRate = 1 - hitRate$

- Hit time
 - Tempo de hit τ_h
 - Tempo necessário para acessar um bloco de memória no nível alto, considerando ainda o custo para verificar se temos ou não um hit
- Miss penalty
 - Penalidade de miss τ_m
 - Tempo necessário para carregar/substituir um bloco do nível mais alto por um do nível logo abaixo

Nesse contexto, consideraremos cache a memória que está no nível mais alto, No entanto, em um âmbito mais geral, cache também pode se referir a qualquer sistema de armazenamento que se beneficia da localidade de acesso

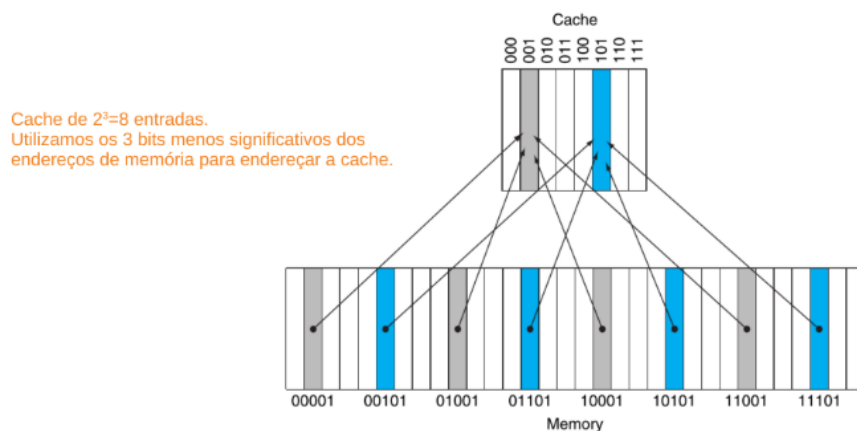
Cache diretamente mapeado

Vamos considerar que: τ

- O processador sempre requisita uma palavra τ
 - 32 bits no MIPS de nossas aulas τ
- Cada bloco da cache também armazena uma palavra
- Quanto o processador requisita uma palavra X_n que não está na cache:
 - Temos um miss τ
 - A palavra então é carregada para a cache

Como encontrar um item nessa cache?

- Cada endereço de memória é mapeado para exatamente uma posição na cache τ
 - Mas cada posição da cache pode ser responsável por um subconjunto de vários endereços da memória principal
- Construímos nossa cache para suportar uma potência de 2 de palavras
 - O mapeamento se torna trivial: τ
 - Para uma cache de 2^n palavras, utilizamos os n bits menos significativos dos endereços para mapear a cache



- Considere mais uma vez que o endereço 00101 é solicitado
- O processador busca o dado no endereço 101 da cache
- Qual palavra da memória está nesse endereço?
- 00101, 01101, 10101, 11101

Tag

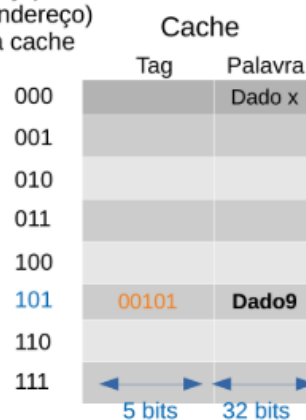
- Além da palavra, cada endereço da cache deve conter o tag da palavra
 - O tag são os bits mais altos da palavra, que não foram utilizados para mapear a cache

Palavras de 32 bits

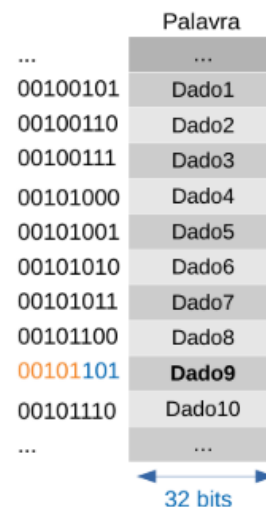
Cache de 8 palavras (3 bits para endereço)

Memória de 256 palavras (8 bits para endereço)

O dado no endereço 00101101₂ está na cache



Memória Principal



Validade da cache

- Imagine que acabamos de ligar o computador e a cache pe inicializada com lixo
- Adicionamos ainda um bit de validade para cada entrada da cache
 - Se o bit é 0, devemos desconsiderar a cache

	valido	Tag	Palavra
000	1	00101	Dado x
001	0	lixo	lixo
010	0	lixo	lixo
011	1	11111	Dado y
100	1	00000	Dado z
101	1	00101	Dado w
110	0	lixo	lixo
111	1	10101	Dado k

Hit quando:

- Utiliza os bits mais baixos de X para encontrar a entrada na cache
- Os demais bits de X são comparados com o tag
- O bit de validade é comparado

Caso contrário, temos um miss:

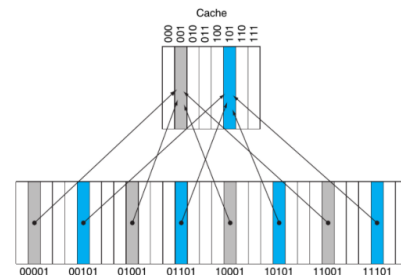
- A palavra é solicitada do nível de memória inferior, e é carregada para o endereço de memória correto $\hat{d}$
 - Caso já haja algo nesse endereço da cache, o dado é substituído
 - O campo tag é atualizado $\hat{d}$
 - O bit de validade é setado
- Finalmente a palavra é enviada ao processador

Blocos de Cache e Associativade

Cache com blocos de uma palavra

- Cada posição (bloco) da cache, armazena exatamente uma palavra

Localidade temporal: Carregamos o dado para a cache, e se no futuro (próximo) ele for necessário novamente, ele já está na cache



Localidade espacial: Única vantagem da localidade espacial é que vizinhos não concorrem pela mesma posição na cache

Mapeamento por blocos

- Vamos dividir a memória em blocos de n bytes
- Considerando o exemplo que cada endereço da memória suporta 1 byte, e cada bloco possui 8 bytes

Bloco	Endereço	Dado
0000 ₂	0000 0000	Dado 0
	0000 0001	Dado 1
	0000 0010	Dado 2
	0000 0011	Dado 3
	0000 0100	Dado 4
	0000 0101	Dado 5
	0000 0110	Dado 6
	0000 0111	Dado 7
0001 ₂	0000 1000	Dado 8
	0000 1001	Dado 9
	0000 1010	Dado 10
	0000 1011	Dado 11
	0000 1100	Dado 12
	0000 1101	Dado 13
	0000 1110	Dado 14
	0000 1111	Dado 15
00010 ₂	0001 0000	Dado 16
...	...	...

Como obter o endereço de bloco?

$$\text{Bloco} = \frac{\text{Endereço}}{\text{Tamanho do Bloco}}$$

- Quando o processador solicita o dado em determinado endereço
 - Verifica o endereço do bloco
 - Utiliza os bits mais baixo do bloco para procurar na cache

- Os bits mais altos do bloco são comparados com a tag

Em caso de hit:

- O bloco está na cache ☑
- Mas o processador não solicitou um bloco, mas sim o dado em um endereço de memória específico
- Os bits que foram descartados para se obter o endereço do bloco, podem ser usados para se obter um “deslocamento dentro do bloco”
 - Offset

Em caso de miss:

- Buscamos todo o bloco da memória e carregamos para a cache

Exemplo:

- Se a CPU solicita 0000 1011 ☑
 - Está no bloco 000012 ☑
 - Deslocado 0112 dentro desse bloco

Memória Principal		
Bloco	Endereço	Dado
0000 ₂	▲ 0000 0000	Dado 0
	0000 0001	Dado 1
	0000 0010	Dado 2
	0000 0011	Dado 3
	0000 0100	Dado 4
	0000 0101	Dado 5
	0000 0110	Dado 6
	▼ 0000 0111	Dado 7
0001 ₂	▲ 0000 1000	Dado 8
	0000 1001	Dado 9
	0000 1010	Dado 10
	0000 1011	Dado 11
	0000 1100	Dado 12
	0000 1101	Dado 13
	0000 1110	Dado 14
	▼ 0000 1111	Dado 15
00010 ₂	▲ 0001 0000	Dado 16
...	...	...

Vantagens

- Aumentamos a localidade espacial ☑
 - Carregamos o dado e seus vizinhos em caso de miss

Desvantagens

- Aumentamos a competição na cache
- A penalidade de falta (miss penalty) se torna maior ☑
 - Precisamos carregar mais dados da memória principal

Tratando escritas

- Quando uma instrução escrever algo, vamos escrever apenas na cache
- Agora para o mesmo endereço de memória, temos dois dados diferentes
 - Cache atualizado
 - Memória de nível mais baixo desatualizado
 - memória fica inconsistente

Write-Through

- Sempre propagar escritas para os níveis mais baixos de memória
- Problemas?
 - Toda escrita deve ser propagada para os níveis mais lentos
 - Precisamos esperar os níveis mais lentos terminarem a operação

Write-Back

- Escrevemos apenas na cache
- O dado é atualizado nos níveis mais baixos apenas quando o dado na cache é substituído
- Utilizado na maioria das CPUs atuais

Associatividade da cache

- Na cache de diretamente mapeada, cada bloco da memória pode ser “encaixado” em apenas uma posição na cache
- Se essa posição já estiver ocupada, precisamos substituir seu conteúdo
 - Mesmo que haja várias outras posições livres na cache que poderíamos utilizar
 - Colisão de endereço

Cache totalmente Associativa

- Uma cache totalmente associativa pode carregar um bloco da memória para qualquer bloco da cache
 - Qualquer bloco livre
 - Se a cache estiver totalmente cheia, podemos escolher o bloco “menos útil” para substituir
- Redução de misses
- Quando a CPU solicita um endereço, precisamos procurar na cache toda

Cache associativa por conjunto

- Meio termo entre um modelo totalmente associativo e o diretamente mapeado
- Cache separada em conjuntos
 - Cada conjunto suporta até n blocos
 - Cache associativa de n vias
- Quando a CPU precisa de um dado no endereço X
 - O conjunto que precisa ser pesquisado é fixo
 - Agora é necessário pesquisar só em todos os blocos do conjunto

Exemplo

- Blocos de 4 bytes
- Cache associativa de 2 vias
- Cache com capacidade para armazenar 8 blocos no total
 - $8 \times 4 = 32$ bytes de capacidade para dados

Conjunto	Tag	Dado (Bloco)	Tag	Dado (Bloco)
00 ₂				
01 ₂				
10 ₂				
11 ₂				

- Bloco de 4 bytes: 2 bits menos significativos são offset
- 8 blocos e 2 vias: 4 conjuntos

Onde o byte no endereço 00001001 pode ser mapeado?

- Está no bloco 000010
- 2 bits de offset
- 6 bits de endereço do bloco q
 - 2 bits mapeamento para endereço do conjunto
 - 4 bits para Tag

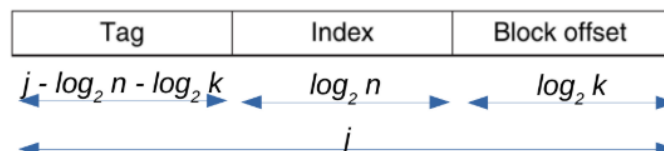
Cache					
Conjunto	V	Tag	Dado (Bloco)	V	Tag
00 ₂	0			0	
01 ₂	0			0	
10 ₂	0		1 0000		
11 ₂	0			0	

Memória Principal		
Bloco	Endereço	Dado(1 byte)
000000 ₂	0000 0000	Dado 0
	0000 0001	Dado 1
	0000 0010	Dado 2
	0000 0011	Dado 3
000001 ₂	0000 0100	Dado 4
	0000 0101	Dado 5
	0000 0110	Dado 6
	0000 0111	Dado 7
000010 ₂	0000 1000	Dado 8
	0000 1001	Dado 9
	0000 1010	Dado 10
	0000 1011	Dado 11
000011 ₂	0000 1100	Dado 12
	0000 1101	Dado 13
	0000 1110	Dado 14
	0000 1111	Dado 15
000100 ₂	0001 0000	Dado 16
	...	...

Endereçamentos com associatividade

k: Número de endereços por bloco

n: Conjuntos



Exemplo

- Considere uma máquina onde a memória é endereçada utilizando 64bits. Considere ainda blocos de tamanho 16, e uma cache associativa de 4 vias com 8 conjuntos
- Quais e quantos bits são utilizados para o offset, o conjunto da cache, e o Tag?



LRU, Caches multinível e Coerência de Cache

Exemplo 1

Considerando uma cache associativa

- ▶ 2 vias
- ▶ Blocos de 4 bytes
- ▶ Capacidade de 64 bytes
- ▶ **Quantos blocos no total?**
 - ★ $\frac{64}{4} = 16$ blocos
- ▶ **Quantos conjuntos no total?**
 - ★ $\frac{16}{2} = 8$ conjuntos
- ▶ **Quantos bits para offset, endereço do conjunto e tag (considerando endereçamentos de 8 e 16 bits)?**
 - ★ $\lg 4 = 2$ bits p/ offset
 - ★ $\lg 8 = 3$ bits p/ endereço do conjunto
 - ★ $8 - 2 - 3 = 3$ bits p/ tag com endereçamentos de 8 bits
 - ★ $16 - 2 - 3 = 11$ bits p/ tag com endereçamentos de 16 bits

Exemplo 2

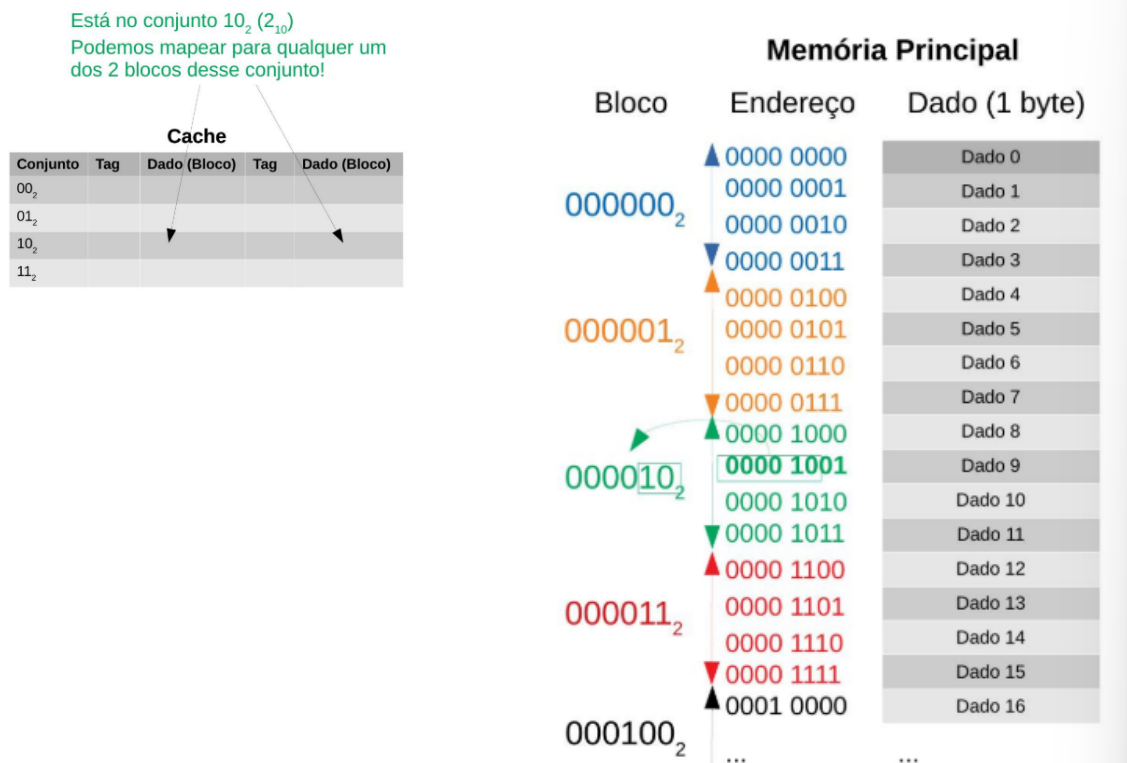
Considerando uma cache associativa

- ▶ 4 vias
- ▶ Blocos de 4 bytes
- ▶ Capacidade de 64 bytes
- ▶ **Quantos blocos no total?**
 - ★ $\frac{64}{4} = 16$ blocos
- ▶ **Quantos conjuntos no total?**
 - ★ $\frac{16}{4} = 4$ conjuntos
- ▶ **Quantos bits para offset, endereço do conjunto e tag (considerando endereçamentos de 8 e 16 bits)?**
 - ★ $\lg 4 = 2$ bits p/ offset
 - ★ $\lg 4 = 2$ bits p/ endereço do conjunto
 - ★ $8 - 2 - 2 = 4$ bits p/ tag com endereçamentos de 8 bits
 - ★ $16 - 2 - 2 = 12$ bits p/ tag com endereçamentos de 16 bits

Exemplo 3

Considerando uma cache associativa

- ▶ 2 vias
- ▶ Blocos de 4 bytes
- ▶ Cache com capacidade para armazenar 8 blocos no total
- ▶ **Qual a capacidade para dados?**
 - ★ $8 \times 4 = 32$ bytes de capacidade para dados
- ▶ **Onde o byte no endereço $0000\ 1001_2$ pode ser mapeado?**



Qual bloco substituir?

- No caso de um miss q
 - Se todos os blocos do conjunto estão ocupados
 - Precisamos substituir um bloco q Qual bloco?

LRU: Lesat recently used

- Remover o bloco que teve o acesso mais antigo

Para esquema de uma cache de 2 vias

- Podemos manter um “bit de uso” em cada bloco do conjunto da cache

- Toda vez que um bloco do conjunto é acessado seu bit de uso é setado 1
- O bit de uso do outro bloco é resetado

Conjunto	V	Uso	Tag	Bloco	V	Uso	Tag	Bloco
0		0				1		
1		1				0		
2		1				0		
3		1				0		

- Mesmo caches simples com apenas 4 vias comumente implementam alguma aproximação do LRU

- Quando um bloco é acessado:
 - O seu bit de uso é setado 1
 - Os bits de uso de todos os demais blocos é resetado 0
 - Agora sabemos qual o bloco usado mais recentemente
 - Mas não sabemos exatamente qual o bloco usado menos recentemente
 - Sabemos apenas que os demais não são a pior escolha possível
 - Nesse caso podemos escolher aleatoriamente entre esses blocos

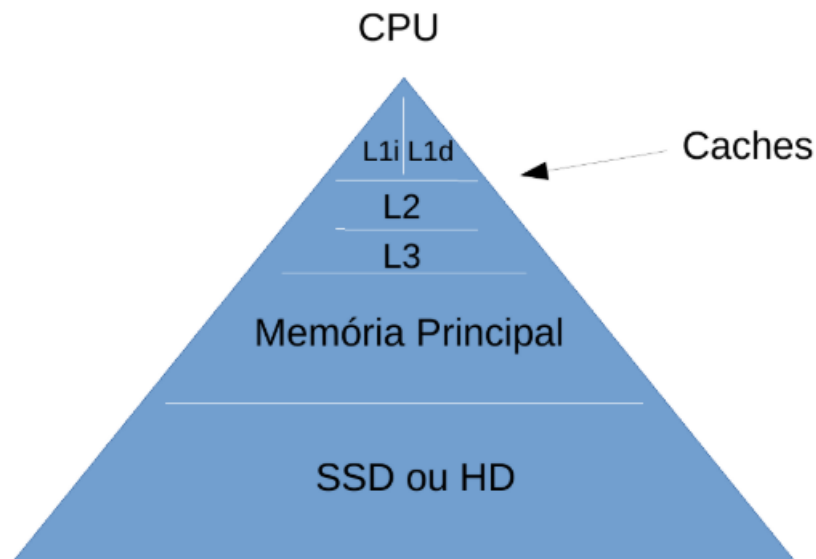
Caches multinível

Níveis mais altos mais próximos da CPU

- Focam na redução do tempo de acesso e custo do miss
 - Como?
 - Reduzir o tamanho do bloco
 - Reduzir o tempo de acesso
- Geralmente de acesso exclusivo para cada núcleo
- Comumente segmentadas entre cache de instrução e cache de dados

Caches menores níveis mais baixos

- Focam na redução da probabilidade de miss
 - Como?
 - Maior associatividade (localidade temporal)
 - Tamanhos de bloco maiores (localidade espacial)
 - Caches maiores
 - Cache compartilhadas entre múltiplos núcleos
- Caches maiores
- Comumente compartilhadas entre os núcleos



Problemas de coerência

- A CPU (Core) 1 solicita um dado da L1
- L1 (do Core 1) não têm o dado (miss) e solicita da L2
- O dado é copiado da L2 para a L1 da CPU 1
- A CPU 1 modifica esse dado
 - O dado é escrito em sua L1, mas não na L2
 - Lembre! Write-back: o dado só vai ser atualizado no outro nível quando o bloco for substituído
- A CPU (Core) 2 solicita o mesmo dado da sua L1
 - L1 (do Core 2) não tem o dado
 - O dado é copiado da L2 para a L1 da CPU 2
 - O dado da L2 está desatualizado!

Protocolo de Snooping

- Quando uma CPU escreve em um bloco da sua cache, envia um sinal para todas as demais CPUs via broadcast para “sujar a cache”
 - As demais CPU’s e níveis de cache desligam o bit de validade do bloco caso elas possuam esse mesmo bloco
 - Se duas CPU’s tentam escrever ao mesmo tempo, uma delas “ganha a corrida” e envia o broadcast antes que a outra
- Quando as outras CPUs precisarem do dado, o bit de validade está desligado
- Efetivamente força que a CPU faça uma nova cópia desse dado
- Agora a cópia é mais complexa
 - Pode vir da cache do vizinho, ou podemos forçar que a CPU que possui a cópia mais recente escreva no nível de baixo para que a CPU vizinha possa enxergar esse dado

- Depende de como implementamos o hardware do processador

Exercício

- Considere que temos blocos de 4 palavras. Considere o bloco a seguir:

Endereço na MP	0000 0000 ₂	0000 0001 ₂	0000 0010 ₂	0000 0011 ₂
Dado	Dado 1	Dado 2	Dado 3	Dado 4

- Se fizermos um programa que executa em paralelo em duas CPUs com a seguinte lógica

CPU 1	CPU 2
Carregue 0000 0000 para reg1	Carregue 0000 0010 para reg1
Carregue 0000 0001 para reg2	Carregue 0000 0011 para reg2
reg3 = reg1 + reg2	reg3 = reg1 + reg2
salve reg3 em 0000 0000	salve reg3 em 0000 0010

- 1 Esses programas compartilham algum dado em algum momento?
- 2 Há risco de uma CPU invalidar os dados da outra?
- 3 Esse programa “paralelo” executa 2x mais rápido do que se fizéssemos tudo sequencialmente em uma única CPU?

Exercício: Respostas

- Os programas não compartilham dados
 - ▶ Enquanto a CPU 1 está trabalhando com os dados 1 e 2, a CPU 2 trabalha com os dados 3 e 4
- No entanto, note que todos os dados estão no mesmo bloco
 - ▶ Quando invalidamos algo na cache, **invalidamos o bloco inteiro**, e não só um pedaço do bloco
 - ▶ Esse problema é chamado de falso compartilhamento
- Esse programa **provavelmente** vai executar mais lento do que se fizéssemos uma versão que utiliza uma única CPU
 - ▶ Na versão atual, uma CPU atrapalha a outra

Exercício

- 1 Para reduzir a probabilidade de falso compartilhamento, o que podemos fazer com o tamanho de blocos na cache?
- Blocos menores reduzem a chance de compartilhamento de variáveis
- Quando você criar um programa, deve também levar em consideração o tamanho dos blocos para evitar essa situação